2026/9/4

LeetCode 字符串学习笔记

学习内容概览

本次整理 `python/leetcode/字符串` 目录中的题目，重点学习字符串处理、字符计数、哈希映射、双指针、原地修改和模式匹配。

题号	题目	核心方法
28	找出字符串中第一个匹配项的下标	字符串查找、滑动比较
49	字母异位词分组	排序字符串、哈希表
58	最后一个单词长度	`strip`、`split`
1502	识别等差数列	排序、相邻差值
242	有效的字母异位词	字符频次统计
283	移动零	快慢指针、原地修改
459	重复的子字符串	枚举周期、字符串重复
709	转换成小写字母	ASCII、字符转换

---

28. 找出字符串中第一个匹配项的下标

思路

给定主串 `haystack` 和模式串 `needle`，寻找模式串第一次出现的位置。源码中先使用了 Python 内置的 `find`，随后实现了直接枚举：从每个可能的起点截取长度为 `m` 的子串并比较。

```python
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        n, m = len(haystack), len(needle)

        for i in range(n - m + 1):
            if haystack[i:i + m] == needle:
                return i
        return -1
```

- 直接比较的时间复杂度：最坏为 $O((n-m+1)m)$。
- 空间复杂度：切片实现通常需要 $O(m)$ 临时空间。
- 实际工程中可使用 `haystack.find(needle)`；如果要求手写高效匹配，可以进一步学习 KMP。

> 循环上界是 `n - m + 1`，因为模式串必须完整落在主串范围内。

49. 字母异位词分组

问题分析

字母异位词由相同字母组成、每个字母出现次数相同，只是排列顺序不同，例如 `eat`、`tea`、`ate`。

源码中最初为每个单词构造字母表并手动比较，同时修改原列表，容易出现索引和删除元素的问题。更简洁的做法是：将单词排序后得到统一的 key，把原单词放入字典对应的分组中。

```
from typing import List


class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        groups = {}

        for word in strs:
            key = "".join(sorted(word))
            groups.setdefault(key, []).append(word)

        return list(groups.values())
```

- 设单词平均长度为 k，总字符数为 N，排序法复杂度约为 $O(N \log k)$。
- 字典保存所有分组，空间复杂度为 $O(N)$。
- 若只含小写英文字母，也可以用 26 位频次数组作为 key，避免排序。

Python 知识点

- `sorted(word)` 返回字符列表，例如 `['a', 'e', 't']`。
- `''.join(sorted(word))` 才能得到字符串 `aet`。
- `join` 只能连接字符串，数字列表需要先使用 `map(str, nums)`。

---

58. 最后一个单词的长度

字符串处理方法

题目要求忽略末尾空格并返回最后一个单词的长度。可以先去除两端空格，再按空格切分，最后读取列表中的最后一项。

```
class Solution:
    def lengthOfLastWord(self, s: str) -> int:
        words = s.strip().split()
        return len(words[-1])
```

- 时间复杂度：$O(n)$。
- 空间复杂度：$O(n)$，因为保存了切分后的单词。

也可以从字符串末尾反向遍历，跳过空格后统计字符数量，这样额外空间为 $O(1)$。

`strip` 和 `split`

- `strip()` 删除字符串开头和结尾的空白字符。
- `strip('abc')` 删除两端连续出现的指定字符集合，不是删除一个固定子串。
- `split()` 默认按空白字符切分，并返回列表。
- `split(' ', 1)` 中的 `1` 表示最多分割一次。

1502. 识别等差数列

错误思路

仅使用等差数列求和公式，根据最小值、最大值和元素总和判断是不充分的。不同的数列可能拥有相同的总和、最小值和最大值，但并不一定是等差数列。

正确思路

先排序数组，再计算相邻两项的差值。之后所有相邻差值都必须与第一项差值相同。

```python
from typing import List


class Solution:
    def canMakeArithmeticProgression(self, arr: List[int]) -> bool:
        arr.sort()
        difference = arr[1] - arr[0]

        for i in range(1, len(arr) - 1):
            if arr[i + 1] - arr[i] != difference:
                return False

        return True
```

- 时间复杂度：$O(n \log n)$。
- 空间复杂度：取决于排序实现；题目允许直接修改数组时可视为额外 $O(1)$。

> 验证结构性质时，不能只验证一个总量公式，还要检查局部相邻关系。

---

242. 有效的字母异位词

频次统计

两个字符串是字母异位词，当且仅当每个字符的出现次数完全相同。源码使用长度为 26 的数组统计小写字母出现次数。

```python
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False

        counts = [0] * 26
        for a, b in zip(s, t):
            counts[ord(a) - ord('a')] += 1
            counts[ord(b) - ord('a')] -= 1

        return all(count == 0 for count in counts)
```

- 时间复杂度：$O(n)$。
- 空间复杂度：$O(1)$，因为统计数组长度固定为 26。

也可以分别统计两个字符串，或直接比较 `sorted(s) == sorted(t)`；排序方法更直观，但复杂度是 $O(n \log n)$。

---

283. 移动零

题目要求

将所有零移动到数组末尾，同时保持非零元素的相对顺序，并且必须在原数组上修改，不能返回一个新数组。

源码中的初版创建了 `res` 并返回它，不符合"原地修改"的要求。

快慢指针

- `fast` 遍历整个数组，寻找非零元素。
- `slow` 指向下一个应该放置非零元素的位置。
- 遍历结束后，将 `[slow:]` 全部设置为 0。

```python
from typing import List


class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        slow = 0

        for fast in range(len(nums)):
            if nums[fast] != 0:
                nums[slow] = nums[fast]
                slow += 1

        for i in range(slow, len(nums)):
            nums[i] = 0
```

也可以在发现非零元素时直接交换 `nums[slow]` 和 `nums[fast]`。两种写法都保持非零元素的相对顺序。

- 时间复杂度：$O(n)$。
- 空间复杂度：$O(1)$。
- 函数按题意不需要返回数组，调用后直接检查原数组即可。

---

459. 重复的子字符串

错误思路

仅统计字符种类数量，再用字符串长度与种类数量的整除关系判断重复是不正确的。字符组成相同并不代表字符排列位置满足周期性，例如 `ababba`。

枚举周期

若整个字符串由长度为 `i` 的子串重复构成，那么字符串总长度必须能被 `i` 整除。满足整除后，再比较：

```
class Solution:
    def repeatedSubstringPattern(self, s: str) -> bool:
        n = len(s)

        for length in range(1, n):
            if n % length == 0:
                unit = s[:length]
                if unit * (n // length) == s:
                    return True

        return False
```

- 时间复杂度：最坏约为 $O(n^2)$。
- 空间复杂度：$O(n)$，包括切片和重复字符串产生的临时对象。
- 更高效的方法可以使用 KMP 的最长前缀后缀数组。

> “字符出现次数满足条件”和“字符排列具有周期”是两种不同性质，不能混为一谈。

---

709. 转换成小写字母

Python 内置方法

实际开发中直接使用 `s.lower()` 最简单。如果练习字符编码，可以利用 ASCII 中大写字母与小写字母相差 32 的规律。

```
class Solution:
    def toLowerCase(self, s: str) -> str:
        result = []

        for char in s:
            if 'A' <= char <= 'Z':
                result.append(chr(ord(char) + 32))
```

```
            else:
                result.append(char)

        return ''.join(result)
```

- 大写字母 ASCII 范围是 `[65, 90]`。
- 小写字母 ASCII 范围是 `[97, 122]`。
- 使用列表收集字符，最后 `join`，比循环中不断拼接字符串更适合较长输入。
- 时间复杂度：$O(n)$；空间复杂度：$O(n)$。

字符串题通用方法总结

1. 字符计数

异位词判断和异位词分组都可以转化为“字符出现次数是否相同”。字符范围固定时，数组比字典更轻量；字符范围不固定时，可以使用字典或 `Counter`。

2. 哈希表分组

设计一个稳定的 key，就能把复杂的两两比较转化为一次遍历：

- 字母异位词：排序后的字符串或频次元组。
- 同一 key 的字符串自然进入同一分组。

3. 双指针与原地修改

移动零体现了“快指针扫描、慢指针维护结果区间”的模式。凡是要求保持相对顺序并在原数组上操作，都可以优先考虑这种方法。

4. 周期与模式匹配

重复子串问题必须同时满足“长度整除”和“内容重复”。只检查长度、字符种类或总和都不足以证明字符串具有周期。

5. Python 字符串工具

方法	作用
`lower()` / `upper()`	大小写转换
`strip()`	删除两端空白或指定字符
`split()`	按分隔符切分为列表
`find()`	查找子串首次出现位置
`sorted()`	返回排序后的新列表
`join()`	使用指定分隔符连接字符串序列

6. 本次学习中的主要易错点

- 不能用总和公式替代对等差结构的完整检查。
- 不能把字符种类数量当成重复子串长度。
- `list` 不能直接作为 `set` 元素或字典 key；需要转换为元组或字符串。
- 题目要求原地修改时，返回新数组不符合要求。
- 修改列表时删除元素可能导致索引跳过，应尽量避免在遍历过程中改变遍历对象。
- `join` 的输入必须是字符串序列。